Students Information

Student Name: كلنا سكشن CS2

1) زياد محمد السيد احمد جاسر

2) عبدالرحمن عاطف سالم

3)زياد احمد ثابت

Problem Information

Problem Title:

Parallel Region (Thread Creation).

Lecture Number:

Lecture 1

Problem Description:

This program demonstrates how OpenMP creates multiple threads and allows each thread to execute the same code block concurrently.

Parallelization Approach:

The OpenMP parallel directive is used to create a team of threads. Each thread obtains and prints its own thread ID using omp_get_thread_num().

Solution Explanation:

A parallel region is defined using #pragma omp parallel. Every created thread executes the enclosed code and prints its unique identifier.

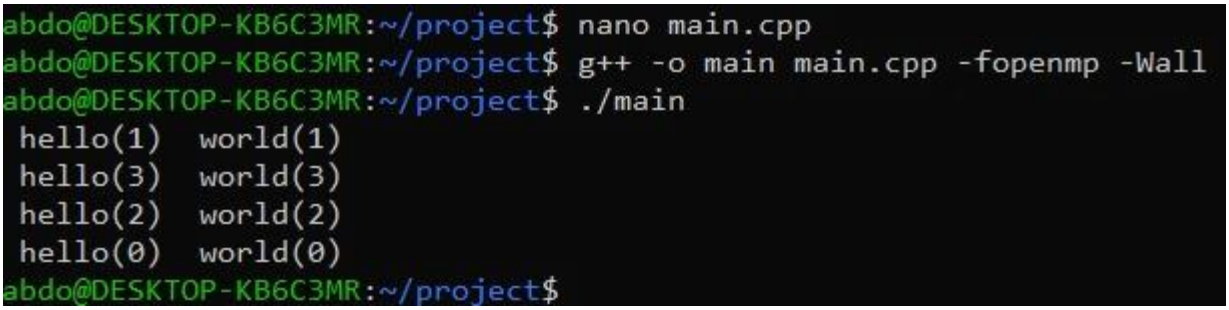
Source Code:

```
#pragma omp parallel
{
```

```
    int ID = omp_get_thread_num();
    printf(" hello(%d) ", ID);
    printf(" world(%d) \n", ID);
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates thread creation and execution in OpenMP using a parallel region.

Problem Information
Problem Title:
Parallel Region with Fixed Number of Threads.

Lecture Number:
Lecture 1

Problem Description:
This program demonstrates how to create a parallel region with a specified number of threads using OpenMP. Each thread executes the same function and processes a shared array.

Parallelization Approach:
The OpenMP parallel directive is used to create four threads. The number of threads is

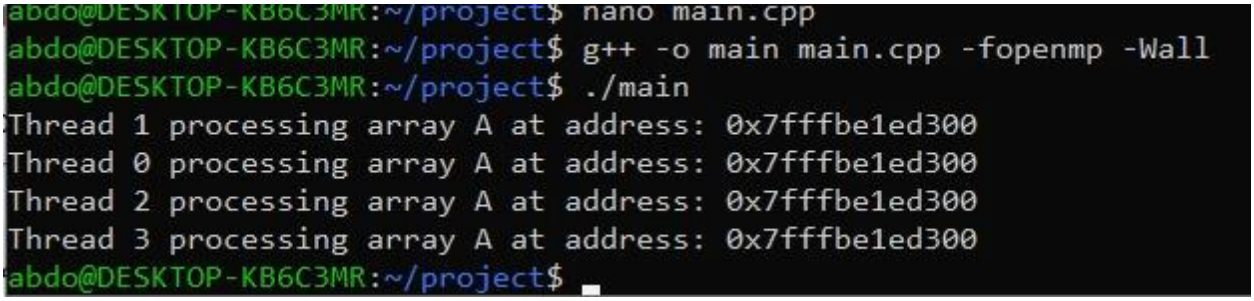
explicitly set using omp_set_num_threads(4). Each thread obtains its ID and calls the pooh() function.

Solution Explanation:
An array is declared and shared among all threads. OpenMP creates four threads, and each thread retrieves its unique ID using omp_get_thread_num(). The thread ID and the address of the shared array are then passed to the pooh() function for processing.

Source Code:

```
#include "omp.h"

#include <stdio.h>

void pooh(int thread_id, double *array) {

  // A sample implementation of the lecture's dummy function

  printf("Thread %d processing array A at address: %p\n", thread_id, (void*)array);

}

int main()

{

    // here I set the max number of threads to 4

    double A[1000] ;

    omp_set_num_threads(4);

    #pragma omp parallel

    {

        int ID = omp_get_thread_num();

        pooh(ID, A);

    }

    return 0 ;

}
```

Execution Results:
Output screenshot attached.



Conclusion

The program successfully demonstrates controlling the number of threads in OpenMP and shows how multiple threads can access and work with the same shared data structure.

Problem Information
Problem Title:
Parallel PI Program (SPMD).

Lecture Number:
Lecture 1

Problem Description:
This program computes the value of π (Pi) using numerical integration. The workload is divided among multiple threads to perform the computation in parallel.

Parallelization Approach:
The program follows the SPMD (Single Program Multiple Data) model. Each thread processes a subset of the iterations and stores its partial sum independently before the final result is combined.

Solution Explanation:
The interval is divided into a large number of steps. Each thread calculates part of the integral by processing iterations assigned to its thread ID. A padded two-dimensional array is used to store partial sums and reduce cache-line contention between threads. After all threads finish, the partial sums are combined to compute the final value of π.

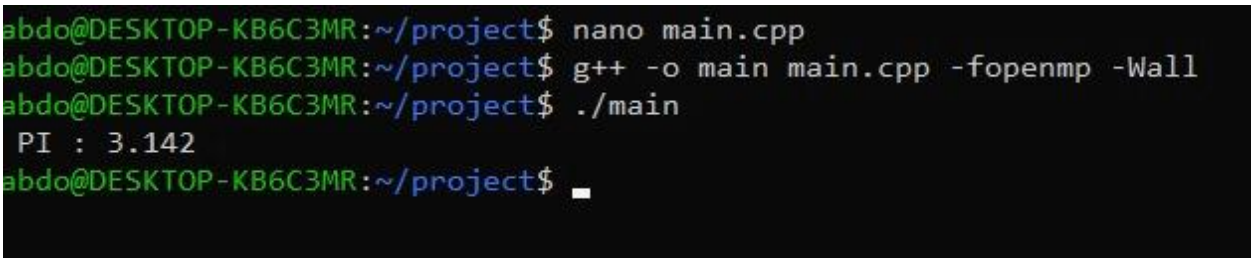
Source Code:

```
#include <omp.h>

#include <iostream>

#include <iomanip>
```

```
static long num_steps = 100000;
double step;
#define  PAD    8       // assume 64 byte L1 cache line size
#define NUM_THREADS 2

using namespace std ;
int main ()
{
      int i, nthreads;  double pi, sum[NUM_THREADS][PAD];
      step = 1.0/(double) num_steps;
      omp_set_num_threads(NUM_THREADS);
      #pragma omp parallel
      {
            int i, id,nthrds;
            double x;
            id = omp_get_thread_num();
            nthrds = omp_get_num_threads();
            if (id == 0)  nthreads = nthrds;
            for (i=id, sum[id][0]=0.0;i< num_steps; i=i+nthrds) {
                  x = (i+0.5)*step;
                  sum[id][0] += 4.0/(1.0+x*x);
            }
      }
      for(i=0, pi=0.0;i<nthreads;i++)pi += sum[i][0] * step;
      cout<< fixed << setprecision(3) << " PI : " <<   pi << endl;
      return 0 ;
```

}

Execution Results:
Output screenshot attached.



Conclusion

The program successfully computes π using parallel execution. By distributing iterations among threads and using padding to reduce cache contention, the parallel implementation achieves better performance than the sequential approach.

Problem Information

Problem Title:

Critical Section.

Lecture Number:

Lecture 2

Problem Description:

This program demonstrates the use of a critical section in OpenMP to safely update a shared variable from multiple threads.

Parallelization Approach:

The workload is distributed among threads using a parallel region. Each thread processes a subset of iterations and updates the shared result variable inside a critical section to prevent race conditions.

Solution Explanation:

Each thread performs a computation using the big_job() function and then processes the result using consume(). Since the variable res is shared among all threads, the update operation is protected by #pragma omp critical to ensure that only one thread modifies it at a time.

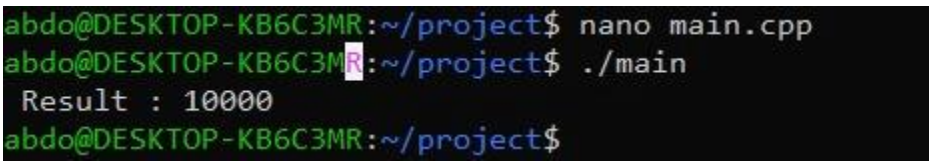
Source Code:

```
#include <omp.h>
#include <iostream>

using namespace std;

const int niters = 100;

float big_job(int i) {
   return (float)(i) * 2.0f;
}

float consume(float B) {
   return B + 1.0f;
}

int main()
{
   float res;
```

```
  #pragma omp parallel
  {
    float B;
    int i, id, nthrds;

    id = omp_get_thread_num();
    nthrds = omp_get_num_threads();

    for(i = id; i < niters; i += nthrds) {
      B = big_job(i);

      #pragma omp critical
      res += consume(B);
    }
  }

  cout << " Result : " << res << endl;
  return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates the use of a critical section to protect shared data. The critical directive prevents race conditions and ensures correct updates to the shared result variable.

Problem Information

Problem Title:

False Sharing Fix Using Critical Section.

Lecture Number:

Lecture 2

Problem Description:

This program computes the value of π (Pi) using numerical integration in parallel. Each thread calculates a local partial sum and contributes its result to the final value.

Parallelization Approach:

The computation is divided among multiple threads using OpenMP. Each thread processes a subset of iterations independently and stores the result in a local variable. The final accumulation is protected using a critical section.

Solution Explanation:

The interval is divided into a large number of steps. Each thread computes its local sum without accessing shared data during the calculation phase. After finishing its assigned iterations, the thread enters a critical section and adds its local result to the shared variable pi. This approach reduces contention during computation and avoids race conditions during accumulation.

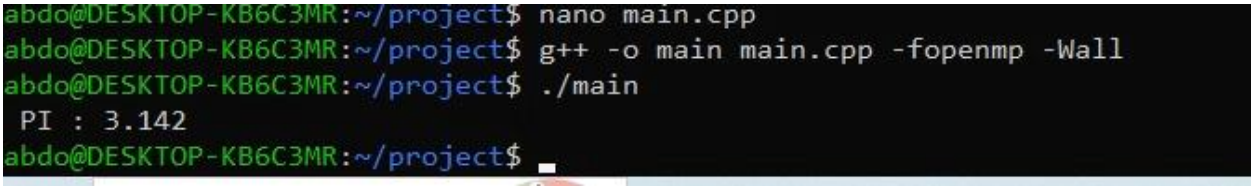
Source Code:

```
#include <omp.h>
#include <iostream>
#include <iomanip>

static long num_steps = 100000;
double step;

#define NUM_THREADS 2

using namespace std;

int main()
{
   double pi;
   step = 1.0 / (double)num_steps;

   omp_set_num_threads(NUM_THREADS);

   #pragma omp parallel
   {
      int i, id, nthrds;
      double x, sum;
```

```
      id = omp_get_thread_num();
      nthrds = omp_get_num_threads();

      if (id == 0)
         nthreads = nthrds;

      id = omp_get_thread_num();
      nthrds = omp_get_num_threads();

      for (i = id, sum = 0.0; i < num_steps; i = i + nthreads) {
         x = (i + 0.5) * step;
         sum += 4.0 / (1.0 + x * x);
      }

      #pragma omp critical
      pi += sum * step;
   }

   cout << fixed << setprecision(3)
       << " PI : " << pi << endl;

   return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully computes π in parallel while avoiding race conditions. Using a local accumulator for each thread reduces shared-memory contention, and the critical section ensures safe accumulation of the final result.

Problem Information

Problem Title:

Parallel PI Program (SPMD).

Lecture Number:

Lecture 1

Problem Description:

This program computes the value of π (Pi) using numerical integration. The computation is divided among multiple threads to improve performance.

Parallelization Approach:

The program follows the SPMD (Single Program Multiple Data) model. Each thread executes the same code but processes a different subset of iterations. Partial results are stored in separate elements of the shared array.

Solution Explanation:

The integration interval is divided into a large number of steps. Each thread calculates a local partial sum and stores it in its corresponding position in the sum array. After all threads finish execution, the main thread combines the partial sums to obtain the final approximation of π.

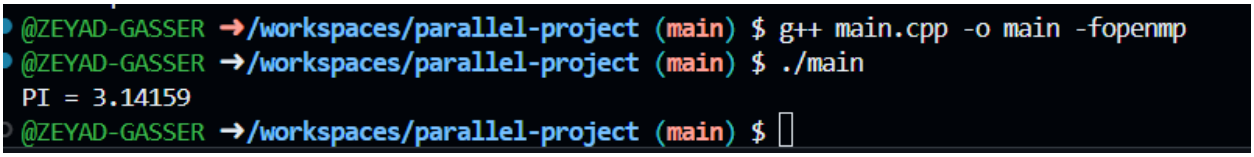
Source Code:

```
#include <omp.h>
#include <iostream>

static long num_steps = 100000;
#define NUM_THREADS 2

int main()
{
   int i, nthreads;
   double pi, sum[NUM_THREADS];
   double step = 1.0 / (double)num_steps;

   omp_set_num_threads(NUM_THREADS);

#pragma omp parallel
   {
      int id, nthrds;
      double x;

      id = omp_get_thread_num();
```

```
        nthrds = omp_get_num_threads();

        sum[id] = 0.0;

        for (int i = id; i < num_steps; i += nthrds)
        {
            x = (i + 0.5) * step;
            sum[id] += 4.0 / (1.0 + x * x);
        }

        if (id == 0)
            nthreads = nthrds;
    }

    pi = 0.0;

    for (i = 0; i < nthreads; i++)
        pi += sum[i];

    pi *= step;

    std::cout << "PI = " << pi << std::endl;

    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully computes π using parallel execution. Each thread calculates a portion of the workload independently, and the final result is obtained by combining all partial sums.

Problem Information

Problem Title:

Parallel For Loop.

Lecture Number:

Lecture 3

Problem Description:

This program demonstrates the use of the OpenMP parallel for directive to distribute loop iterations automatically among multiple threads.

Parallelization Approach:

The loop iterations are divided automatically between the available threads using the #pragma omp parallel for directive. Each thread processes a subset of the iterations independently.

Solution Explanation:

An array of size MAX is initialized in parallel. For each iteration, a value is calculated and passed to the big() function. The returned result is stored in the corresponding array element. Since each thread writes to a different array index, no synchronization is required.

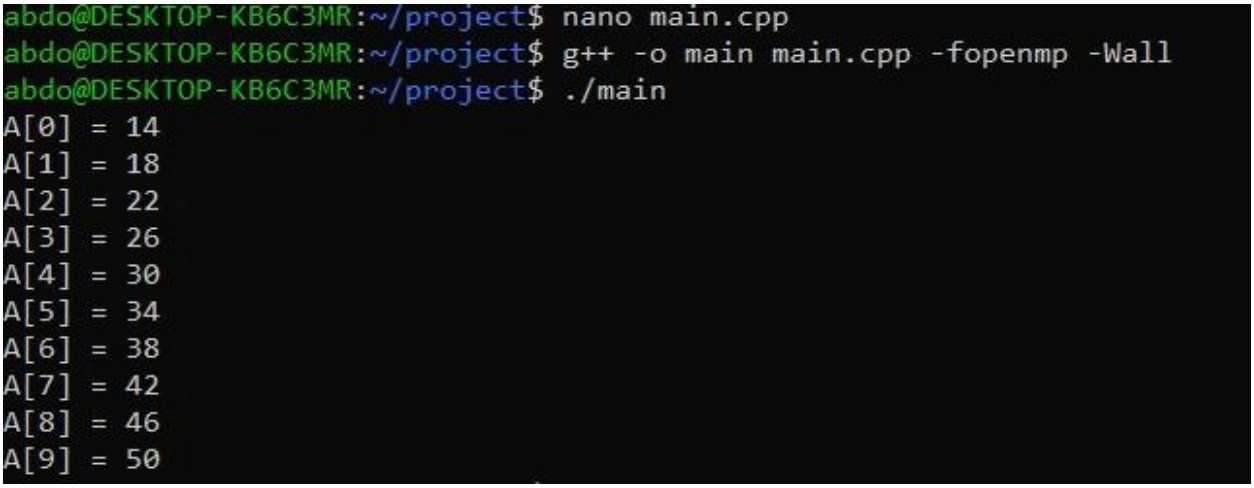
Source Code:

```
#include <omp.h>
#include <iostream>

using namespace std;

const int MAX = 10;

int big(int j) {
  return j * 2;
}

int main() {
  int A[MAX];

  #pragma omp parallel for
  for (int i = 0; i < MAX; i++) {
    int j = 5 + 2 * (i + 1);
    A[i] = big(j);
  }

  for (int i = 0; i < MAX; i++) {
```

```
    cout << "A[" << i << "] = " << A[i] << "\n";
  }


  return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates automatic loop distribution using OpenMP. The parallel for directive simplifies parallel programming by assigning iterations to threads without requiring manual workload management.

Problem Information

Problem Title:

Reduction Clause.

Lecture Number:

Lecture 3

Problem Description:

This program demonstrates the use of the OpenMP reduction clause to perform parallel accumulation of array elements and calculate their average value.

Parallelization Approach:

The array summation is parallelized using the #pragma omp parallel for directive with a reduction operation on the variable ave. Each thread computes a partial sum, and OpenMP combines all partial results automatically at the end of the loop.

Solution Explanation:

The array is initialized with values from 1 to 100. The parallel loop calculates the sum of all elements using the reduction clause. After the summation is completed, the total sum is divided by the number of elements to obtain the average.

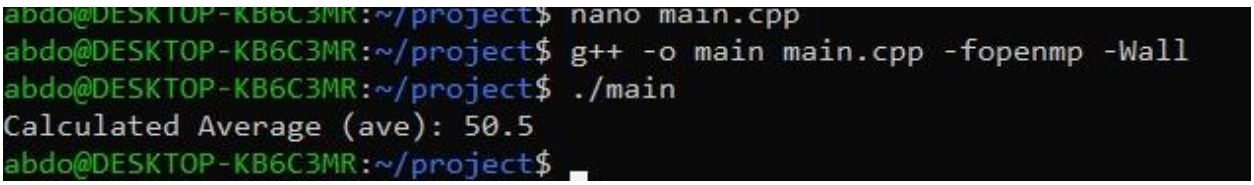
Source Code:

```
#include <omp.h>
#include <iostream>

const int MAX = 100;
using namespace std;

int main() {
  double ave = 0.0;
  double A[MAX];
```

```
    // initialize the array elements with dummy data
    for (int i = 0; i < MAX; i++) {
        A[i] = static_cast<double>(i + 1);
    }

    // parallel loop with an addition reduction on the 'ave' variable
    #pragma omp parallel for reduction(+:ave)
    for (int i = 0; i < MAX; i++) {
        ave += A[i];
    }

    // calculate the final average value
    ave = ave / MAX;

    cout << "Calculated Average (ave): " << ave << "\n";

    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates the reduction clause in OpenMP. The reduction operation allows multiple threads to safely accumulate values into a single variable without requiring explicit synchronization.

Problem Information

Problem Title:

Barrier Synchronization.

Lecture Number:

Lecture 3

Problem Description:

This program demonstrates the use of barrier synchronization in OpenMP to ensure correct execution order between dependent computations in parallel threads.

Parallelization Approach:

The program uses a parallel region where each thread performs an initial computation independently. A barrier is then used to ensure that all threads complete the first phase before moving to the second phase, which depends on shared data.

Solution Explanation:

Each thread first computes a value and stores it in the shared array A. After the barrier, all threads are guaranteed to have completed writing to A. Then each thread computes a second value using its own and a neighboring thread's data, storing the result in array B.

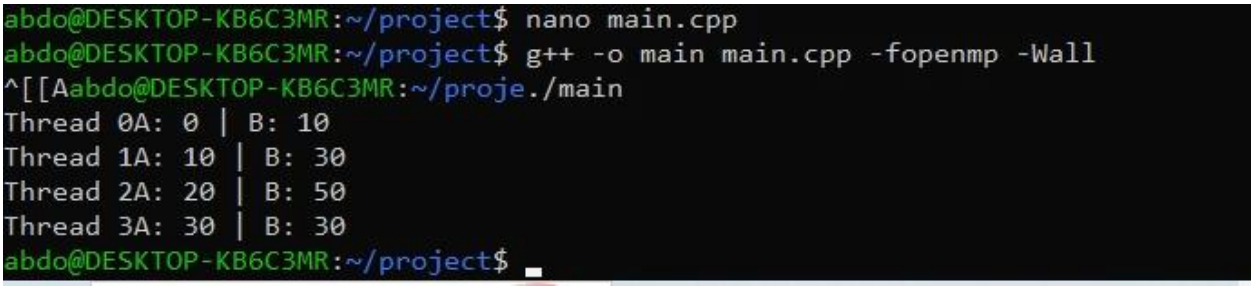
Source Code:

```
#include <iostream>
#include <omp.h>

using namespace std;

const int NUM_THREADS = 4;

int big_calc1(int id) {
  return id * 10;
}

int big_calc2(int id, int* shared_array) {
  int neighbor_id = (id + 1) % NUM_THREADS;
  return shared_array[id] + shared_array[neighbor_id];
}

int main() {
  int A[NUM_THREADS] = {0};
  int B[NUM_THREADS] = {0};

  omp_set_num_threads(NUM_THREADS);

  #pragma omp parallel
  {
    int id = omp_get_thread_num();
```

```
        A[id] = big_calc1(id);

        #pragma omp barrier

        B[id] = big_calc2(id, A);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        cout << "Thread " << i
             << " A: " << A[i]
             << " | B: " << B[i] << "\n";
    }

    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates barrier synchronization in OpenMP. The barrier ensures that all threads complete the first computation phase before any thread proceeds to the second dependent phase.

Problem Information

Problem Title:

Atomic Operation.

Lecture Number:

Lecture 3

Problem Description:

This program demonstrates the use of atomic operations in OpenMP to safely update a shared variable from multiple threads without race conditions.

Parallelization Approach:

The program creates a parallel region where each thread performs independent computations. The final update to the shared variable X is protected using the atomic directive to ensure safe concurrent access.

Solution Explanation:

Each thread computes a temporary value using the DOIT() and big_ugly() functions. Since the variable X is shared among threads, the update operation is performed using #pragma omp atomic to ensure that the addition is executed safely without interference between threads.

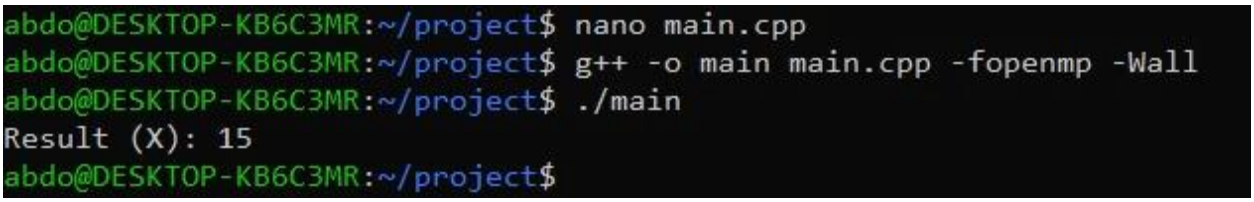
Source Code:

```
#include <iostream>
#include <omp.h>

using namespace std;

// Shared global accumulator variable
double X = 0.0;

double DOIT() {
   return 1.5;
}

double big_ugly(double B) {
   return B * 2.5;
}

int main() {
   omp_set_num_threads(4);

   #pragma omp parallel
   {
      double tmp, B;

      B = DOIT();
      tmp = big_ugly(B);
```

```
        // Atomic synchronization: safely update shared variable
        #pragma omp atomic
        X += tmp;
    }

    cout << "Result (X): " << X << "\n";
    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates the atomic directive in OpenMP. It ensures that updates to the shared variable are performed safely without race conditions while maintaining parallel execution efficiency.

Problem Information

Problem Title:

Parallel For Loop .

Lecture Number:

Lecture 2

Problem Description:

This program demonstrates how OpenMP distributes loop iterations among multiple threads using the parallel for directive, while safely printing thread execution details.

Parallelization Approach:

The program uses a parallel region combined with an OpenMP for loop to distribute iterations across threads. Each iteration is processed by a different thread. A critical section is used to ensure that output to the console is not interleaved.

Solution Explanation:

A parallel region is created with 4 threads. The loop iterations from 0 to N-1 are automatically divided among these threads. Each thread calls the NEST_STUFF() function, which prints the thread ID and the iteration index. The critical section ensures that printing is performed safely without overlapping outputs.

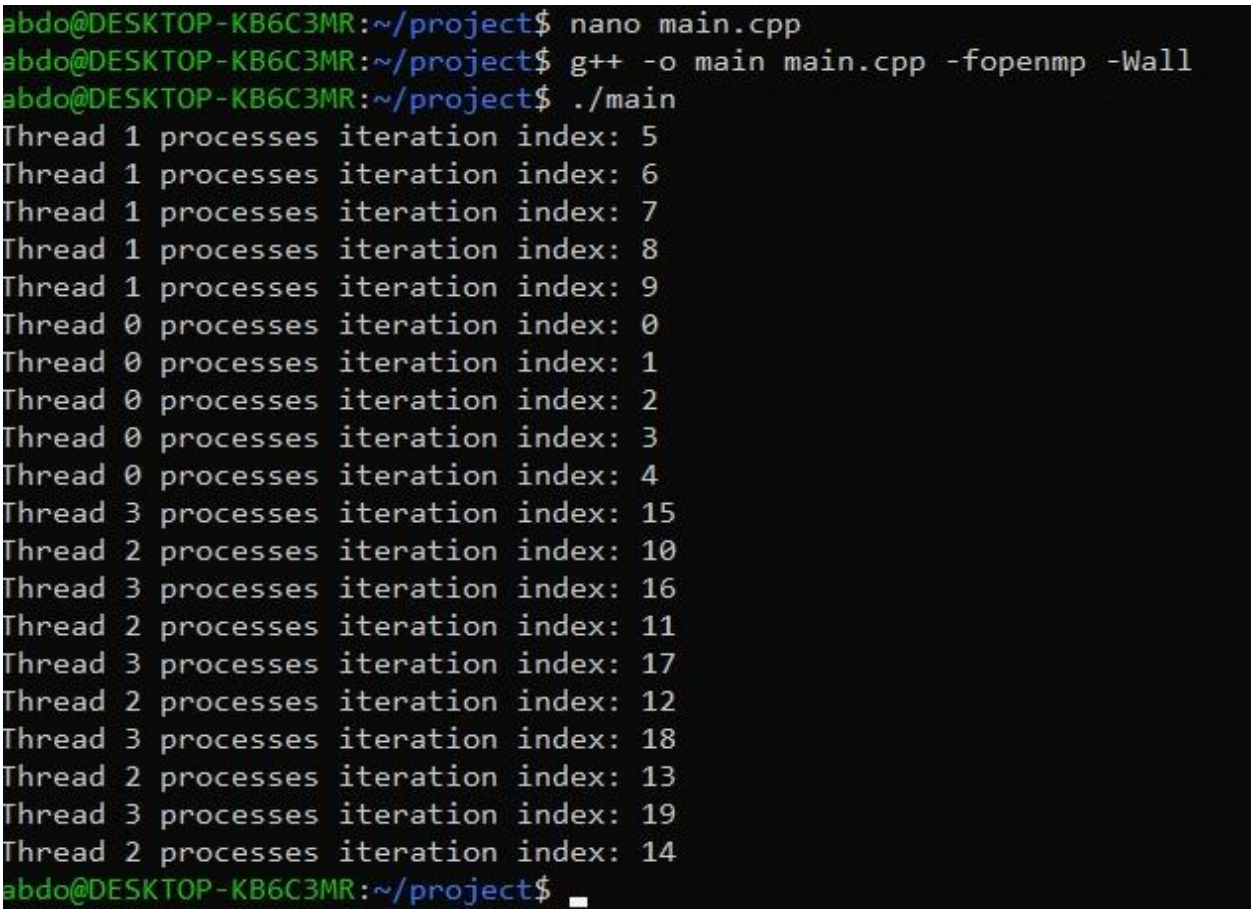
Source Code:

```
#include <iostream>
#include <omp.h>

using namespace std;

const int N = 20;

void NEST_STUFF(int i) {
   // prints the mapping of loop iterations to specific system threads
```

```
    #pragma omp critical
    {
        cout << "Thread " << omp_get_thread_num()
             << " processes iteration index: " << i << "\n";
    }
}

int main() {
    omp_set_num_threads(4);

    #pragma omp parallel
    {
        #pragma omp for
        for (int i = 0; i < N; i++) {
            NEST_STUFF(i);
        }
    }

    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates loop parallelization using OpenMP. The for directive distributes iterations among threads, and the critical section ensures correct and readable output formatting.

Problem Information

Problem Title:

Nested Loop Parallelization (Collapse Clause).

Lecture Number:

Lecture 4

Problem Description:

This program demonstrates how OpenMP parallelizes nested loops using the collapse clause to improve workload distribution across multiple threads.

Parallelization Approach:

The program uses the #pragma omp parallel for collapse(2) directive to combine two nested loops into a single iteration space. This allows OpenMP to distribute all iterations of both loops evenly among available threads.

Solution Explanation:

Two nested loops iterate over a 3x3 matrix. The collapse(2) clause merges both loops so that iterations are treated as a single loop. Each thread processes different matrix elements, and a critical section is used to safely print the output to avoid mixed console output.

Source Code:

```
#include <iostream>
#include <omp.h>

using namespace std;

const int N = 3;
const int M = 3;

int main() {
   omp_set_num_threads(4);
```

```
    #pragma omp parallel for collapse(2)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {

            #pragma omp critical
            {
                cout << "Thread " << omp_get_thread_num()
                    << " handles matrix element: ["
                    << i << "][" << j << "]\n";
            }
        }
    }

    return 0;
}
```

Execution Results:

Output screenshot attached.

```
abdo@DESKTOP-KB6C3MR:~/project$ nano main.cpp
abdo@DESKTOP-KB6C3MR:~/project$ g++ -o main main.cpp -fopenmp -Wall
abdo@DESKTOP-KB6C3MR:~/project$ ./main
Thread 0 handles matrix element: [0][0]
Thread 0 handles matrix element: [0][1]
Thread 0 handles matrix element: [0][2]
Thread 2 handles matrix element: [1][2]
Thread 2 handles matrix element: [2][0]
Thread 3 handles matrix element: [2][1]
Thread 1 handles matrix element: [1][0]
Thread 3 handles matrix element: [2][2]
Thread 1 handles matrix element: [1][1]
abdo@DESKTOP-KB6C3MR:~/project$
```

Conclusion

The program successfully demonstrates nested loop parallelization using the collapse clause in OpenMP. It improves workload distribution by flattening nested loops and assigning iterations efficiently across threads.

Problem Information

Problem Title:

Communication Bandwidth Measurement Program (MPI Point-to-Point Communication).

Lecture Number:

Lecture 5

Problem Description:

This program measures the communication performance between two MPI processes by sending messages of different sizes and calculating the average transfer time and bandwidth.

Parallelization Approach:

The program uses MPI with two processes (rank 0 and rank 1). Process 0 acts as the sender and process 1 acts as the receiver. Multiple message sizes are tested to evaluate communication performance under different workloads.

Solution Explanation:

The program allocates a buffer of varying sizes and performs repeated MPI_Send and MPI_Recv operations. The sender measures the total communication time using MPI_Wtime(), and the receiver simply receives the messages. The average time per transfer is computed, and bandwidth is calculated based on message size and transfer time.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define NUMBER_OF_TESTS 10

int main(int argc, char* argv[])
{
   double *buf;
   int rank, numprocs;
   int n;
   double t1, t2;
   int j, k, nloop;
   MPI_Status status;
```

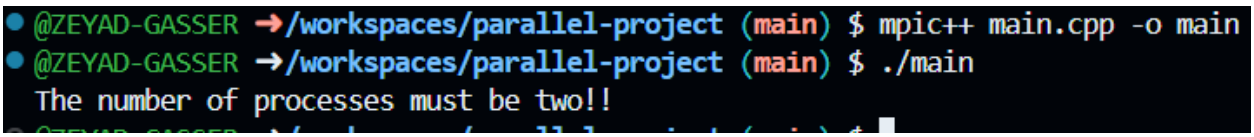
```
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &numprocs);

if (numprocs != 2)
{
   printf("The number of processes must be two!!\n");
   MPI_Finalize();
   return 0;
}

MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (rank == 0)
{
   printf("\tn\ttime [sec]\tRate [Mb/sec]\n");
}

for (n = 1; n < 100000000; n *= 2)
{
   nloop = 1000000 / n;

   if (nloop < 1)
      nloop = 1;

   buf = (double *)malloc(n * sizeof(double));
```

```
if (!buf)
{
   printf("Could not allocate message buffer of size %d\n", n);
   MPI_Abort(MPI_COMM_WORLD, 1);
}

for (k = 0; k < NUMBER_OF_TESTS; k++)
{
   if (rank == 0)
   {
      t1 = MPI_Wtime();

      for (j = 0; j < nloop; j++)
      {
         MPI_Send(buf, n, MPI_DOUBLE, 1, k, MPI_COMM_WORLD);
      }

      t2 = (MPI_Wtime() - t1) / nloop;
   }
   else if (rank == 1)
   {
      for (j = 0; j < nloop; j++)
      {
         MPI_Recv(buf, n, MPI_DOUBLE, 0, k,
               MPI_COMM_WORLD, &status);
      }
```

```
        }
    }

    if (rank == 0)
    {
        double bandwidth;
        bandwidth = n * sizeof(double) * 1.0e-6 * 8 / t2;

        printf("\t%10d\t%10.8f\t%8.2f\n",
             n, t2, bandwidth);
    }

    free(buf);
  }

  MPI_Finalize();
  return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully evaluates MPI communication performance by measuring latency and bandwidth across different message sizes. It demonstrates point-to-point communication between two processes using MPI_Send and MPI_Recv.

Problem Information

Problem Title:

Ping-Pong Message Transfer (MPI Point-to-Point Communication).

Lecture Number:

Lecture 5

Problem Description:

This program demonstrates point-to-point communication between two MPI processes using a ping-pong message exchange with increasing message sizes.

Parallelization Approach:

The program uses two MPI processes. Process 0 sends a message to process 1, and process 1 replies back to process 0. This exchange is repeated for different message sizes to observe communication behavior.

Solution Explanation:

The message size starts from a small value and is doubled in each iteration. Process 0 initiates communication by sending a buffer to process 1. Process 1 receives the message and sends it back to process 0. This back-and-forth exchange continues until the maximum message size is reached.

Source Code:

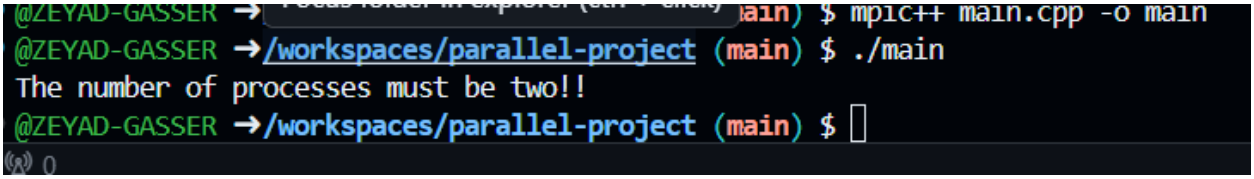
```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char* argv[])
{
   int numprocs, rank, tag = 100, msg_size = 64;
   char *buf;
   MPI_Status status;

   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);

   if (numprocs != 2)
   {
      printf("The number of processes must be two!!\n");
      MPI_Finalize();
      return 0;
   }

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   printf("MPI process %d started...\n", rank);
   fflush(stdout);
```

```
while (msg_size < 10000000)
{
   msg_size *= 2;

   buf = (char *)malloc(msg_size * sizeof(char));

   if (rank == 0)
   {
      MPI_Send(buf, msg_size, MPI_BYTE, 1, tag, MPI_COMM_WORLD);
      printf("P0 sent message size %d to P1\n", msg_size);
      fflush(stdout);

      MPI_Recv(buf, msg_size, MPI_BYTE, 1, tag, MPI_COMM_WORLD, &status);
   }

   else if (rank == 1)
   {
      MPI_Recv(buf, msg_size, MPI_BYTE, 0, tag, MPI_COMM_WORLD, &status);
      MPI_Send(buf, msg_size, MPI_BYTE, 0, tag, MPI_COMM_WORLD);

      printf("P1 exchanged message size %d with P0\n", msg_size);
      fflush(stdout);
   }

   free(buf);
}
```

```
    MPI_Finalize();
    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates ping-pong communication between two MPI processes. It highlights bidirectional message exchange and shows how communication behavior changes as message size increases.

Problem Information

Problem Title:

Parallel π Program (Broadcast and Reduction Communication).

Lecture Number:

Lecture 6

Problem Description:

This program computes the value of π (Pi) using numerical integration in a distributed memory environment with MPI. The computation is parallelized across multiple processes.

Parallelization Approach:

The program uses MPI collective communication operations. The input value (number of intervals) is broadcast from process 0 to all other processes using MPI_Bcast. Each process computes a partial sum independently, and the final result is combined using MPI_Reduce.

Solution Explanation:

Process 0 reads the number of intervals and broadcasts it to all processes. Each process computes its portion of the integration range using a cyclic distribution. The partial results are then reduced (summed) into the root process, which computes the final approximation of π and measures execution time.

Source Code:

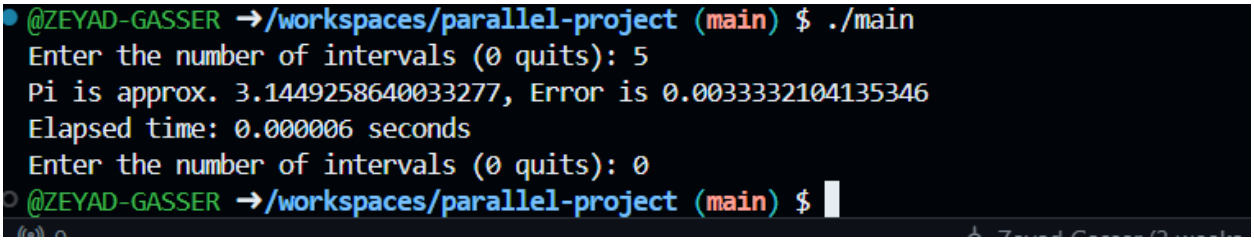
```
#include <stdio.h>
#include <math.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
  int done = 0, n = 0, myid, numprocs, i;
  double PI25DT = 3.14159265358979323846264 3;
  double pi = 0.0, h, sum, x, start = 0.0, finish;

  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
  MPI_Comm_rank(MPI_COMM_WORLD, &myid);
```

```
while (!done)
{
   if (myid == 0)
   {
      printf("Enter the number of intervals (0 quits): ");
      fflush(stdout);

      if (scanf("%d", &n) != 1)
         n = 0;

      start = MPI_Wtime();
   }

   MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);

   if (n == 0)
   {
      done = 1;
   }
   else
   {
      h = 1.0 / (double)n;
      sum = 0.0;

      // Cyclic distribution of workload
      for (i = myid + 1; i <= n; i += numprocs)
```

```
        {
          x = h * ((double)i - 0.5);
          sum += 4.0 / (1.0 + x * x);
        }

        MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

        if (myid == 0)
        {
          finish = MPI_Wtime();
          printf("Pi is approx. %.16f, Error is %.16f\n",
                pi * h, fabs(pi * h - PI25DT));

          printf("Elapsed time: %f seconds\n", finish - start);
        }
      }
    }

    MPI_Finalize();
    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates MPI collective communication using broadcast and reduction. It efficiently distributes the computation of π across multiple processes and aggregates the final result in the root process.

Problem Information

Problem Title:

Communicator Splitting (Process Group Management).

Lecture Number:

Lecture 6

Problem Description:

This program demonstrates how MPI processes can be divided into separate groups using communicator splitting, and how each group can perform independent collective operations.

Parallelization Approach:

The program uses MPI_Comm_split to divide the global communicator into two subgroups based on the parity of the process rank (even and odd). Each subgroup then works independently with its own communicator.

Solution Explanation:

Each process determines its new rank and group using MPI_Comm_split. After splitting, each subgroup computes the sum of ranks using MPI_Reduce. The root process of each group prints the result, showing independent group-level communication.

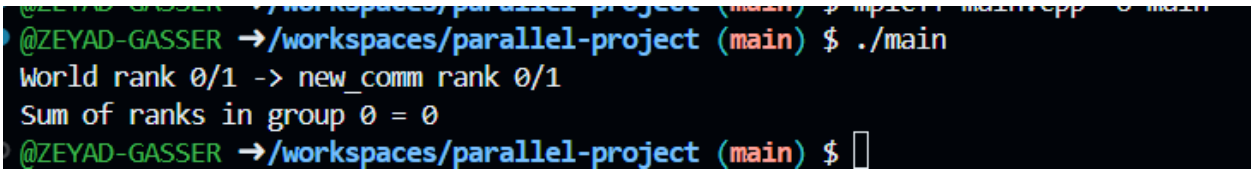
Source Code:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv)
{
   int numprocs, org_rank, new_size, new_rank;
   MPI_Comm new_comm;

   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &org_rank);

   // Split processes into even / odd groups
   MPI_Comm_split(MPI_COMM_WORLD, org_rank % 2, 0, &new_comm);

   MPI_Comm_size(new_comm, &new_size);
   MPI_Comm_rank(new_comm, &new_rank);

   printf("World rank %d/%d -> new_comm rank %d/%d\n",
```

```
        org_rank, numprocs, new_rank, new_size);

    int sum_ranks = 0;

    // Reduce ranks inside each group
    MPI_Reduce(&new_rank, &sum_ranks, 1, MPI_INT, MPI_SUM, 0, new_comm);

    if (new_rank == 0)
    {
        printf("Sum of ranks in group %d = %d\n",
            (org_rank % 2), sum_ranks);
    }

    MPI_Comm_free(&new_comm);
    MPI_Finalize();

    return 0;
}
```

Execution Results:

Output screenshot attached.



Conclusion

The program successfully demonstrates MPI communicator splitting, where processes are divided into independent groups. Each group performs its own collective communication, showing process group management in MPI.
